CyPhyHouse: A programming framework for rapid development of distributed mobile robotic applications

Ritwika Ghosh, Joao Paulo Jansch-Porto, Peter Du, Amelia Gosse, Sayan Mitra, and Geir Dullerud
{rghosh9,janschp2,peterdu2,gosse2,mitras,dullerud}@illinois.edu
Coordinated Science Laboratory
University of Illinois at Urbana-Champaign
Urbana, IL 61801

***Abstract*— This paper introduces a new programming and software development framework—CyPhyHouse—for distributed mobile robotics. CyPhyHouse comes with a programming language (Koord), a compiler, a simulator, and a verification tool. Users can develop applications on the high-level, event driven *Koord language* without fretting about low-level protocols. This application program can be simulated on dozens of robots using the *simulator*; the same code can be compiled and deployed on heterogeneous mobile platforms, and the application code can also be verified through bounded model-checking using the *verification tool*. In this paper, we describe the different aspects of CyPhyHouse through two simple applications and discuss the technical challenges, experiences, and design decisions.**

I. Introduction

Consider the development process for a location-sharing application for mobile phones. The developer codes in a high-level language like Objective C or Swift, focusing her energies on getting the application logic right. A compiler generates the executable code automatically. Even executables for different hardware platforms can be generated automatically, essentially by linking the correct platform-specific libraries. The programming and compiler infrastructure comes with tools for testing, debugging, and validation. Due to this software framework, the simple app can be written, deployed, and tested over a weekend, by high school students.

Contrast that with the development of a similar location-based application for a heterogeneous collection of mobile robots. There are no languages for programming robots that give the programmer the high-level of abstractions to say, for example, "move from A to B avoiding U". ROS [1] provides many useful packages that work across different platforms, but it does not provide needed programming abstractions nor the compiler support. There is also a paucity of tools for deploying the generated code on the actual robots. Integrated, testing and validation tools for distributed robotic systems are also currently not available. Consequently, developing and debugging the application, even based on textbook algorithms, will take weeks. Since heterogeneous robotic platforms can be used for many tasks like mapping, surveillance, delivery, coordination, the absence of the development ecosystem means that each application has to be developed from scratch with little reuse, modularity, or portability.

In this paper, we address this gap by introducing *CyPhyHouse*[1]: an open source software framework for programming, rapid deployment, and testing of distributed robotics applications. The high-level *Koord* language enables users to write succinct distributed coordination applications without getting bogged down by messaging and thread management issues (See Examples in Figure 3 and Figure 5). Using the CyPhyHouse framework around *Koord*, a user can code, compile, launch, and run applications in a highly-automated fashion (Figure 1). The framework has been built over three years and has more than 100k lines of source code. The

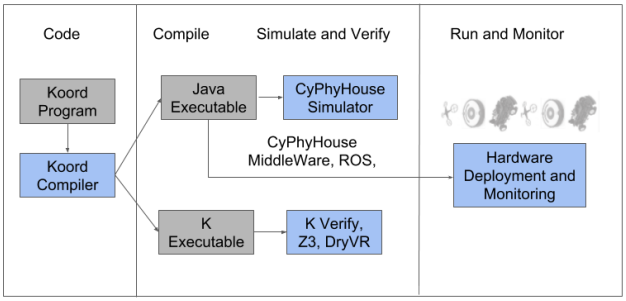

Fig. 1: CyPhyHouse framework. Major tools shown in blue.

technical contributions of this paper are:

- Design and development of the CyPhyHouse open source software system. This includes a discrete event simulator for distributed robotic systems, the application launcher, the run-time logging and monitoring system, and an integrated indoor positioning system. All of these software tools are integrated with our new robot programming language called *Koord* and its compiler.
- First demonstration of the feasibility of end-to-end high-level application development and deployment on a distributed, heterogeneous, platform involving quadcopters and ground vehicles.
- Demonstration of the feasibility of system-level verification of applications written in *Koord*.

Design and development of the *Koord* programming language and the supporting $\mathbb{K}$-based [2] verification tool *Ko-*

[1] https://cyphyhouse.github.io

ordBMC are discussed here for the sake of completeness; those details will appear elsewhere [3].

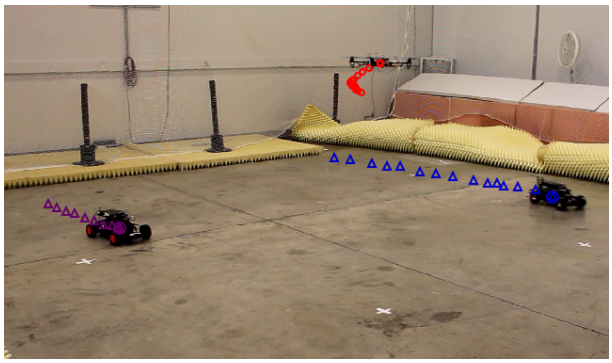

Fig. 2: Experimental run in our testbed. The traces show the path of each robot for the last 2 seconds.

Related Work: Modern programming languages like C# and Swift, and compiler infrastructures like LLVM [4] have revolutionized the application development ecosystem in mobile computing. Inspired by these successes, there is a surge of interest in open and portable languages that raise the level of abstraction [5], [6], [7], [8] (For an earlier survey of Domain Specific Programming Languages (DSLs) for robotic systems see [9]. Most of these older languages are proprietary or generate executable files that are tied to specific platforms). Buzz [5] and React [7] fall in this category as does our language *Koord*. The Live Robot Programming language [10] not only provides a higher-level programming abstraction in terms of nested state machines, but also allows the program to be changed while running, hence reducing the feedback loop across writing, compiling, and testing of robot programs. Buzz currently does not connect with verification tools, and the verification approach implemented with React uses precise models of the environment and performs model checking using dReal [11]. In contrast, our environment implementations allow for imprecise models, with a verification approach using sensitivity analysis [12]. The Robotarium project provides remote end-to-end access to a multi-robot research facility, but not languages and development tools [13]. The VeriPhy project [6] shares a similar goal to CyPhyHouse; however, instead of a programming language, the starting point is differential dynamic logic [14], and there are significant differences in the underlying verification engines used (KeYmaera X, HOL instead of K, Z3, DryVR).

"Correct-by-construction" synthesis from high-level temporal logic specifications has been applied to mobile robotic systems (see, for example [15], [16], [17], [18], [19]). Our point of view on automating robot programming is different in that we expect that the programmer's creativity and efforts will be necessary well beyond writing high-level specs in solving distributed robotics problems; consequently only the tedious and standard steps in coordination and control are automated using the *Koord* compiler.

Programming systems using the shared memory paradigm have been developed for several distributed computing systems [20], [21], [22], [23], [24]. Specifically, P [25] and PSync [26] are DSLs for asynchronous partially distributed systems, but cyber-physical interactions are not supported. *Koord* provides a distributed shared variable abstraction for coordinating multiple agents. The consistency semantics implemented here are that the writes to shared variables are propagated to all the agents, and become visible to other agents reading the variable after one round (δ time units). In the fault-free synchronous model considered here a gossip algorithm-based is used to implement this semantics in the *Koord* runtime system.

II. Overview of Building Apps using CyPhyHouse

Building a new CyPhyHouse application involves three easy steps: code, compile, and run. We describe the software suite we have developed for supporting each step.

A. High-level coding for robotics with Koord

The user writes the application code using our new high-level programming language: *Koord*. We have developed *Koord* as an event-triggered, distributed shared memory-based programming language that provides abstract constructs for robot motion control. The *Koord* program *Lineform* for a set of 10 robots to form a equi-spaced line is shown in Figure 3. Notice how close this program is to a textbook description of a consensus protocol. The user can then: compile *Lineform* to Java and simulate it for scenarios with dozens of robots; compile it with ROS bindings and deploy it on different robot platforms by simply editing a configuration file; and compile it to a $\mathbb{K}$ executable [2] and verify bounded time safety properties, e.g., that the robots do not exit the convex hull of their initial positions.

Reading Koord*:* Robot pid periodically sets its destination to be the average of the positions of $pid - 1$ and $pid + 1$. If all robots (except the extremal ones) repeat this operation periodically, then they would eventually form an equi-spaced line. Thanks to the shared variable x that *Koord* allows to declare in our *Lineform* program, the positions of the neighboring robots are available to read at robot pid, which in turn makes this program succinct yet understandable.

```
Lineform
  using Motion actuator target sensor pos, status
  allread: Position ⟨∗⟩ x

  Update:
    eff x[pid] = Motion.pos
      if (pid ≠ 1 or pid ≠ 10)
        Motion.target = (x[pid + 1] + x[pid − 1])/2
```

Fig. 3: Line formation program written in *Koord* (*Left*) and its mathematical counterpart in robotics and control textbooks (*Right*).

Koord Apps*:* *Lineform* is a simple application. In Section III we discuss a more interesting application (*Task*) in which a collection of robots have to coordinate and visit a sequence of points in a mutually exclusive and exhaustive fashion. Task app is a building-block for collaborative mapping, delivery, inspection, fire-fighting, etc. Other *Koord*

apps include a smart intersection, a distributed landing protocol inspired by NASA's SATS [27], and a platooning application for automated vehicles. For a deep dive into syntax, semantics, and pragmatics of *Koord*, we have the language design document [3].

B. Compiling to run, simulate, or verify applications

We have developed a *Koord* compiler that automatically generates executables from the high-level application code (Figure 3), which can be deployed on different platforms. Currently implemented support includes quadcopters, Roombas, and F1/10 RC cars [28]. The compilation process has a few stages. First, a (platform independent) Java executable is generated. This executable is then linked with platform dependent libraries, ROS drivers, and low-level controllers. For example, different path-planners and low-level controllers are linked for quadrotors, than those that are used for RC cars. The resulting linked executables are then ready to be uploaded on the individual robots.

The platform independent executable can also be simulated using the *CyPhyHouse simulator* (see, Figure 4). The simulator runs the actual executable but simulates the motion of the robots according to specified physics models. Thus, the CyPhyHouse simulator is a very general discrete event simulator for multi-agent systems, with 2D-visualization, and provides a clear and quick way of testing applications on a large number of heterogeneous robots (See more details in Section III-C).

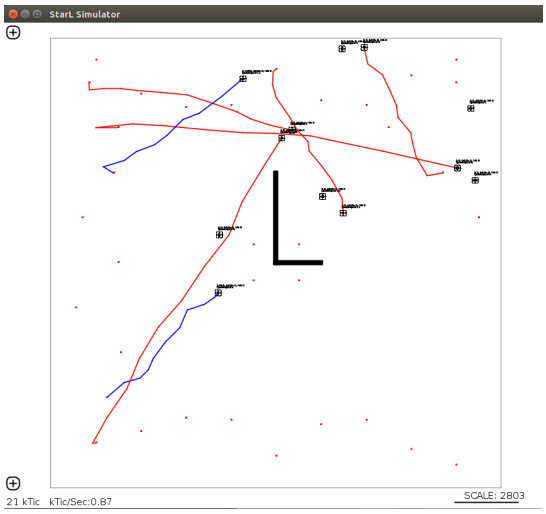

Fig. 4: CyPhyHouse simulator screenshot showing distributed collision-free path planning in *Task* app with static and dynamic obstacles. Blue: collision-free paths planned to waypoints; red: possibly conflicting paths. In this application, a robot with the blocked path will either compute a new destination or wait for red path to be cleared. "L": static obstacle.

Finally, the we have developed the complete formal semantics of *Koord* in the $\mathbb{K}$ framework [2]. This means that we can formally verify the high-level application code against required invariant properties. For example, the *KoordBMC* tool we have developed can automatically check that the system of robots executing the *Lineform* application satisfies the invariant "$\forall i \in ID, x[0] \leq x[i] \leq x[10]$" which asserts that all the robots stay within the segment defined by robots 0 and 10. Verification of distributed systems is a fundamentally challenging problem with very few tools and algorithms that scale [29]. Verification of applications controlling continuous motion makes this into a hybrid verification problem, which is even more challenging. Our verification approach combines the formal semantics of $\mathbb{K}$ with state-of-the art hybrid verification tool DryVR [12] to show the feasibility of code-level verification of distributed hybrid systems. In Section III we present a summary of our verification techniques. Once again, we refer the reader to [] for technical details.

C. Real-time monitoring and running applications

When the application is executing on the platforms, the CyPhyHouse runtime framework records detailed timed logs including program states, events, positions, sensor data, message queues, etc. These logs can be visualized and annotated for the purpose of debugging. We have also developed a simple 3D, visualization for real-time monitoring and for users to provide inputs to the applications.

III. The CyPhyHouse Development Framework

In this section we provide more details on the software development environment. We begin with the *Koord* programming language and discuss a nontrivial application, then we discuss verification, simulation, and hardware deployment.

A. Distributed Task Allocation in Koord

We will describe key features of *Koord* using a coordination application which we will also use later in experiments. An agent in the distributed system consists of a) a *Koord program*, b) a *controller*, and c) a physical plant . The program and the controller interact with each other through *sensor* and *actuator* ports (Figure 1). The program reads the sensor ports, reads and writes to program variables, and writes to actuator ports—represented in *Koord* as *events* which may occur if their preconditions hold true. The controller drives the underlying physical plant. The *environment* of a *Koord* program is the combination of the controller and the plant. The set of agents in the system may interact with each other using *shared variables*. Each agent in the system runs an instance of the *same Koord* program.

Task Coordination Application: Figure 5 shows a distributed task allocation application written in *Koord*. Distributed task allocation is challenging as it involves mutual exclusion as well as construction of de-conflicted paths. Tasks are abstractions for real location-based objectives like package delivery, surveillance, or fire-fighting. Koord enables implementation of simple task allocation strategies using shared variables.

In *Task* app, the agents maintain a shared list of tasks (*taskList*) to complete, and each agent is uniquely assigned a task from this list in the event *Assign*. Once an agent reaches the task location (event *Reached*), it does whatever the task entails (event *Complete*), and continues this process.

We use the modular nature of *Koord* to implement dynamic path planning, mutual exclusion through shared variables, and collision avoidance.

The *Task* app uses a controller called *Motion* which is expected to drive the robots through a route, as directed by the position value set at its actuator port *Motion.route*. The sensor ports that *Motion* makes available to the *Task* program are: i) *Motion.psn*: the robot position in a fixed coordinate system (provided by either GPS or an indoor positioning system) ii) *Motion.done*: A flag indicating whether the *Motion* controller is active, or finished reaching the previous *route* target points.The implementation of *Motion* controller may involve, for example, path planners, and platform-specific steering and throttle controllers, and drivers for specific positioning systems.

Variables and Functions: Aside from local program variables each agent program also has access to two simulation parameters (a) a unique integer identifier *pid* for itself and (b) a list *ID* of identifiers of all participating agents. [2]

Koord allows use of developer provided functions with the modules that are being used in an application. The user can also write their own functions allowed by the Koord syntax. *newTask* is a function which creates a task with a unique task id. There are other (library) functions, defined as auxiliary functions in the $\mathbb{K}$ semantics of *Koord*, supporting list comprehension. For instance, *get* is a function that takes as input a list and an index, and returns the element at that index; *getPath* takes as input a location, and invokes a pathplanner that computes a path from the current location of the agent to the provided location; *notNull* is a function which indicates whether a path has an intersection with any of the paths in a given set of routes.

In addition to local program variables, *Koord* programs can use distributed shared variables, for coordination across multiple agents. The *Task* application uses a shared (*allwrite*) list of *tasks* called *taskList*, which can be updated by any participating agent. Mutual exclusion is always an essential feature when shared variables are involved. *Koord* provides a locking mechanism using the keyword *atomic*, which the *Assign* event uses to update the shared list of tasks *taskList* safely.

Koord supports two types of shared variables, multiwriter-multireader, and singlewriter-multireader shared variables. Writes to a shared variable by one agent are propagated by *Koord*'s runtime system, and become visible to other agents in the next round. The semantics of shared variables in *Koord* is designed so that each agent maintains a local copy of the shared variables. The set of all local copies of the variables is seen as the *distributed shared memory*. The correctness of a program relies on consistent values of the agents' local values of shared variables. When an agent writes to a variable, it updates its local copy and the *Koord* framework uses message passing to inform the other agents of the change. Ideally, these changes should occur before the next round of computations begins. When

[2]This set of participating agents is known and constant.

```
1  Task(pid)
2
3  def task :
4     pos loc
5     int asgnid
6     int taskId
7
8  using Motion
9     sensors:‘
10       pos psn,bool done
11    actuators:
12       pos[ ] route
13
14 allwrite:
15    list<task> taskList
16
17 allread:
18    pos[ ] routes[pid ] = Motion.psn
19
20 local:
21    int i = 0
22    path p
23    task temp, new
24    enum Stage {ASGN,RCHD,CMPT} stage = ASGN
25
26 Assign:
27    pre stage == ASGN
28    eff atomic:
29      for (i = 0,i < length(taskList),i = i+1):
30         temp = get(taskList,i)
31         p = getPath(temp.loc)
32         if temp.asgnid == 0 and notNull(p,routes):
33            new = new Task(temp.loc,pid,temp.taskId)
34            put(taskList,i,new)
35            routes[pid ] = p
36            Motion.route = p
37            stage = RCHD
38
39 Reached:
40    pre stage = RCHD and Motion.done
41    eff routes[pid ] = temp.loc
42       stage = CMPLT
43
44 Complete:
45    pre stage == CMPLT
46    eff stage = ASGN
```

Fig. 5: *Koord* program for robot i to do required tasks at provided locations.

an agent reads a variable, it only accesses the local copy. This policy might result in agents using older copies of variables during a program transition, but we demonstrate using our experimental evaluation that our implementation specifications prevent this from occurring (see Section V).

Events: *Task* has 3 events, specified by preconditions(*pre*) and effects(*eff*). A precondition is a predicate that evaluates to true or false, and an effect consists of *statements*. An event is *enabled* when its precondition evaluates to true, and the agent can execute statements contained in an enabled event's effect. This is analogous to a timed automaton taking an enabled action. In *Task*, after being assigned a task in *Assign*, the *Motion* controller uses the *route* set in this event to reach the location of the assigned task. It is possible that no event is enabled while an agent is trying to reach the location of this task, so no event executions happen until the agent reaches its target. After reaching the task location (*Reached*), the agent sets its *stage* to CMPLT, which indicates that an agent is performing a task. After δ time, a new computational round starts, at which point the agent will start to assign a new task location if there are any.

B. Formal verification of Koord applications

We used the $\mathbb{K}$ semantic framework [30] to develop a formal verification tool for *Koord* applications. $\mathbb{K}$ allows users to write mathematical semantic rules which automatically generate an executable compiler for the language which is faithful to the language semantics. We used this executable formal semantics of *Koord* to build *KoordBMC* which is a bounded model checker for *Koord*. The verification results for several applications are shown in Table I. Verification of distributed cyber-physical systems is an extremely challenging problem with only a few tools available. Although, *KoordBMC* currently only scales to small problem instances, in context, this is a promising tool connecting a high-level

TABLE I: **Verification Summary**

Benchmark	Agents	Num. of Rounds	Time to Verify	Safe
HVAC	3	10	2m18s	Safe
HVAC	4	10	2m51s	Safe
HVAC	3	10	38s	Unsafe
HVAC	4	10	43s	Unsafe
Task	2	10	1m8s	Safe
Task	3	10	3m6s	Safe
Task	3	10	4m13s	Unsafe
Task	4	10	8m29s	Unsafe
Fischer's	2	10	3m4s	Safe
Fischer's	2	10	1m19s	Unsafe
Braking	3	10	1m58s	Safe
Braking	4	10	3m29s	Safe
Braking	3	10	1m28s	Unsafe
Braking	4	10	1m43s	Unsafe

language to actual implementations.

KoordBMC uses the formal language semantics to produce an overapproximation of the *states* that the system can reach within the given execution. Here, a state of the system at a given time refers to the values of the program variables, and the controller ports at that time. The input to our verification tool is (i) P: the program, (ii) inv: a candidate invariant function, (iii) ID: set of agents, (iv) δ: time step size and (v) n: length executions to check.

KoordBMC outputs 'safe' if the inv is indeed an n-invariant, 'unsafe' if it finds a counter-example execution of length n. In some cases, it can return 'unknown' due to the fact that we are computing overapproximations of the states of the system, and checking whether they are safe. While the presented results were obtained by using explicit models of the system dynamics, we can also handle black box dynamics by combining the verification module of *KoordBMC* with the sensitivity-based reachability tool DryVR [12].

The *Agents* column in the table refers to the number of agents in the system. The applications are (i) a simple HVAC model that should ensure that the temperature of several roosm should remain above a given temperature, (ii) the above *Task* app, that should ensure that there are no collisions, (iii) Fischer's mutual exclusion protocol, (iv) and a Braking system, that should ensure that a platoon of cars maintains a separation between any consecutive cars. (v) For more details on these benchmarks refer to [3].

C. Simulating *Koord* applications

We developed a simulator Figure 4 for *Koord* applications within the CyPhyHouse framework to enable the user a way of testing their discrete event loop with simple motion models to test and debug the application program logic. The CyPhyHouse simulator uses motion automata, which can be provided any programmable dynamics to simulate the robot dynamics. The simulated sensed data is also provided realistic noises and dynamics to mimic actual system dynamics. The communication protocols are also a part of the simulator communication module. The time between two computational steps (or discrete event loop iterations) is used to propagate messages to implement shared memory as in the actual hardware stack. We present the results of our experiments below in Figure 6.

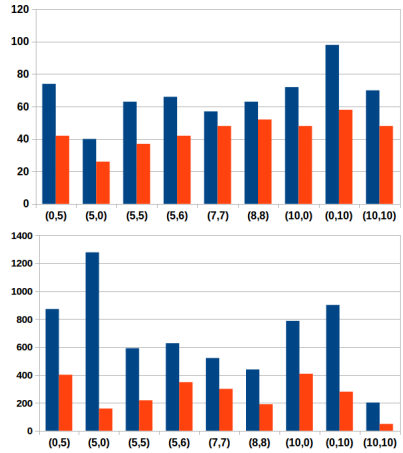

Fig. 6: Performance evaluation of $Task$ app with CyPhyHouse Simulator for 40 task sequence: the x axis refers to the number of agents, with $(cars, quadrotors)$. *Top:* Blue- total completion time and orange- time taken for conflict resolution in seconds *Bottom:* Blue- maximum distance travelled by an agent and orange- minimum in cm.

We observed that conflict resolution took more time with more robots, as expected. The completion time remained relatively stable. The maximum and minimum distances travelled were relatively closer with more robots. However, all these results were affected by the non-determinism in choice of paths and tasks by the agents, and their previous relative positions at the moment of choosing the next task as well. These results demonstrate a capability of covering various scenarios in simulation without having to incur the cost of hardware deployment.

D. CyPhyHouse middleware and sensing interfaces

We developed a software-hardware communication module to support the three-plane architecture of the *Koord* runtime system; which comprises of a program which interacts with the controller plane through sensor and actuator ports, which in turn communicates with the physical plant through ROS [1] messages.

The program plane itself consists of the high-level language interface with middleware that provides the language features. In a nutshell, the runtime system automatically generates code for the application using all the supported libraries, such as the implementation of the distributed shared variables using message passing over WiFi, motion automata of the robots, high-level collision and obstacle avoidance strategies, etc. The *Koord* front-end contains a compiler that takes as input programs written in the *Koord* high-level language and generates Java application code that runs with the Java middleware for the *Koord* framework. The generated Java program is designed to contain a (potentially infinite) discrete event loop consisting of events corresponding to those in the high level program.

Figure 1 includes the detailed structure of the software stack including the interfaces. We use ROS to handle the low-level interfaces with hardware. Therefore, to communicate between the high-level programs and low-level controllers, we need a means of communication between Java and ROS.

The Java middleware programs need to subscribe to existing ROS topics and execute appropriate callback functions upon receiving newly published messages. They should also be able to create ROS topics and publish messages to all existing subscribers. To this end, our implementation of the sensor and actuator interfaces connects the platform related Java code to a ROS Bridge server [31] ensuring that the application can run on hardware without having to worry about compatibility with ROS and vice versa.

Our framework supports middle level path-planning module replacement as well, in addition to low-level waypoint following by the hardware platforms. The choice of path planner used can be platform-specific and/or application-specific. In the current version of the software, we have implemented two different path-planning algorithms: a simple path-planner, which generates a straight path between the current robot position and the destination; and a RRT-based planner, which accounts for possible obstacles in the field and the robot dynamics. Due to the modular structure of the *Koord* system, it is easy for a user to implement a new path planner if a different behavior is desired.

IV. Experimental Testbed and Hardware

So far, we discussed the hardware independent components of the software stack; we now describe the experimental testbed used to validate our claims, along with the interaction of the different platforms with the software stack.

As previously mentioned, the CyPhyHouse framework targets heterogeneous robotics platforms. In order to demonstrate such capabilities, we have built both a car and a quadcopter. Note that the car has nonholonomic constraints, while the quadcopter has uncertain dynamics, so in other standard settings, a roboticist would have to develop a separate application for each platform.

Fig. 7: Our different robotic platforms. *Left:* F1/10 Car, *Right:* Quadcopter. On both vehicles the reflective markers used by the motion capture system are visible.

Quadcopter: The quadcopter was assembled from off-the-shelf hardware, with a 40cm $\times$ 40cm footprint. The main computing unit consists of a Raspberry Pi 3 along with a Navio2 deck for sensing and motor control. Stabilization and reference tracking are handled by Ardupilot [32]. Between the Java layer and Ardupilot we include a hardware abstraction layer (HAL) to convert setpoint messages from the high-level language into MAVLINK using the mavROS library ([33]), so Ardupilot can parse them. The position data can come from any source, as long as all robots use the same coordinate system. In this paper we receive data from a motion capture system. Since the autopilot was originally meant to use a GPS module, we also convert the current quadcopter position into the Geographic Coordinate System before sending it to the controller.

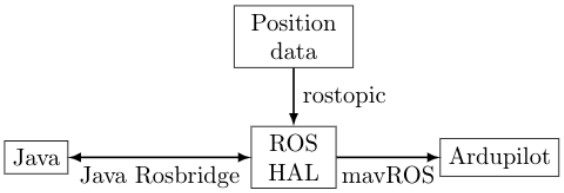

Fig. 8: Diagram of the interaction of the *Koord* software stack with the quadcopter. The ROS HAL node parses both position and setpoint messages into the message format that the Ardupilot uses.

Car: Similar to the quadcopter, the car platform uses off-the-shelf hardware based on the open-source MIT RACECAR project. The computing unit consists of an NVIDIA TX2 board. The car platform has a similar software diagram as the quadcopter, except that the Ardupilot module is removed. The ROS node uses the current position (from any positioning system) and desired waypoints (sent by the Java layer) to compute the input speed and steering angle. The car has an electronic speed controller that handles low-level hardware control.

Test Arena: We performed our experiments in a 7m $\times$ 8m $\times$ 3m arena equipped with 8 Vicon motion capture cameras. The Vicon system allows us to track the position of multiple robots with sub-millimeter accuracy. The reflective markers used by the motion capture system can be seen in both vehicles in Figure 7. The Test Arena is shown in Figure 2

V. Experimental validation

In this section, we carry out case studies in our testbed to demonstrate some of the claims presented in the previous sections. The experiments were carried out in part in the Intelligent Robotics Laboratory, University of Illinois.

A. Experimental Setup

We performed several experiments involving the deployment of the *Task* application on cars and quadcopters (see Section IV for the hardware details). Due to the limited size of the arena, we had to limit the concurrent number of robots. The units of measurement and the coordinate system used by the indoor positioning system, the low-level controllers, and the Koord application program are all provided in meters. Our implementation of the quadcopters's

waypoint follower stipulates that it reaches a location (task) provided it enters a $r = 30$cm ball around the location, and the indoor positioning system observes and transmits that fact. The car is considered to have reached a location if it enters a circle with radius $r = 25$cm around the specified location. This parameter value is decided by the platform developer, and not available to the user in the high level language.

B. Case Studies

Task distribution: Figure 9 (Left) shows the trajectories of a car and a quadcopter executing the *Task* app. The car visits the locations of all the ground level tasks ($z = 0$) and the quadcopter visits the locations of all the other ($z > 0$). To avoid collisions, the quadcopter takes off when the application starts and lands after all the tasks have been completed. We added another car to the set of agents executing this app, requiring more coordination between the agents (as shown in Figure 9 (Right)) that is handled automatically by the language. The user only needs to change the configuration file to reflect this change from 2 agents to 3.

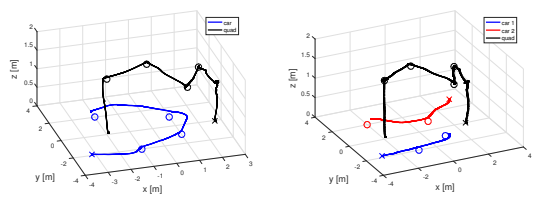

Fig. 9: *Left:* trajectories for 1 car and 1 drone performing *Task*; *Right:* trajectories for 2 cars 1 drone. Circles represent the tasks given, while the crosses show the starting location of each robot.

Blocking behavior: Figure 11 demonstrates the efficacy of the task application itself in performing collision avoidance. For these experiments, we deployed the *Task* app shown in in Figure 5 on two cars, where the agents go to the first available task they can compute a path to. Figure 10 shows the computed (and eventually completed) paths. From Figure 11, we see that car 1 acquires the point at $\vec{x} = (2, -2)$ first and starts moving towards it at $t = 0.38$ s, and car 2 only starts moving once car 1 has reached and released its path from the blocked routes.

Repeated tasks: Here, we want to demonstrate that the system will robustly scale up with a larger list of waypoints while still demonstrating the behavior exemplified in the previous two experiments. Since we have a limited test space, we gave a list of repeating waypoints. The resulting paths for 2 cars can be seen in Figure 12. In order to clearly see the blocking behavior for this experiment, we show a snippet of the positions of the two cars in Figure 13.

VI. Conclusion

In this paper we presented the CyPhyHouse software development and deployment system for distributed robotic applications. It interfaces with and complements existing tools commonly used by roboticists, such as ROS, providing easy integration with popular platforms by almost any user.

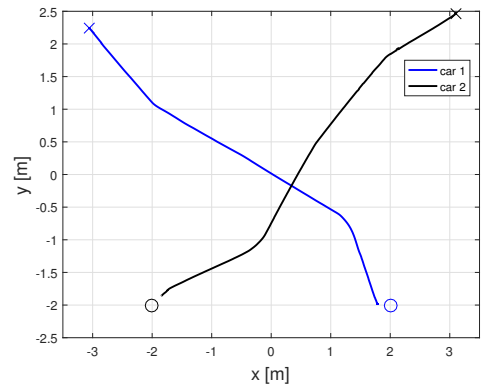

Fig. 10: Trajectories for 2 cars in the *Blocking behavior* experiment

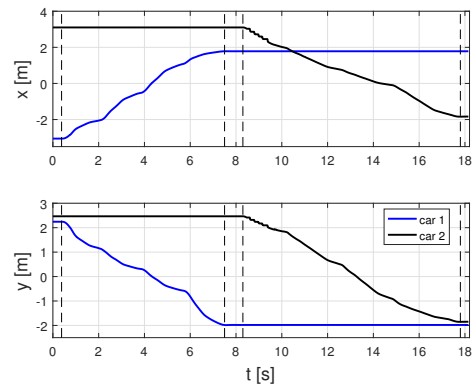

Fig. 11: *Top:* x vs t for blocking paths for two cars. *Bottom:* y vs t for blocking paths for two cars. The dashed lines represent the starting and stopping times for the cars moving on their paths.

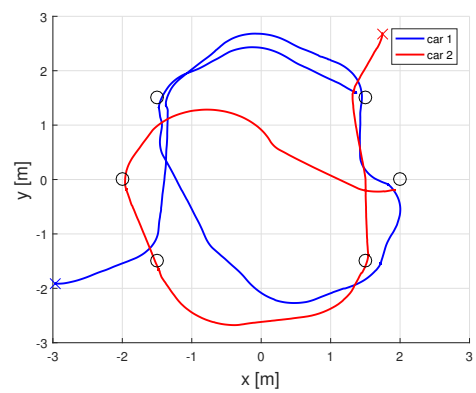

Fig. 12: Trajectory for 2 cars in the *Repeated tasks* experiment.

Our study suggests that *Koord* and the *Koord* compiler can indeed enable users to develop and run distributed robotics applications in a hardware independent fashion; *Koord* programs can be ported across platforms automatically by linking with the right platform dependent libraries; and the *Simulator* and *KoordBMC* tools can be used to simulate and verify the same application code. In the future, we plan to perform a user study to gain a better understanding of the robot app developer's experience and efficiency with

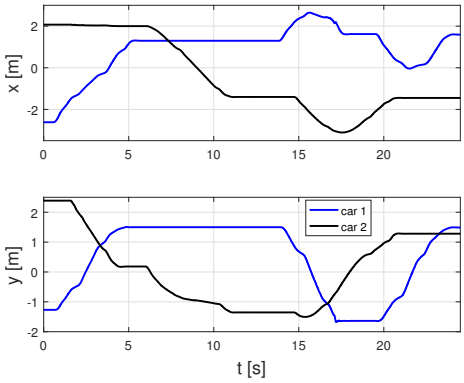

Fig. 13: *Top:* x vs t for *Repeated tasks* for two cars. *Bottom:* y vs t for *Repeated tasks* for two cars.

CyPhyHouse.

References

[1] M. Quigley, K. Conley, B. P. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, and A. Y. Ng, "Ros: an open-source robot operating system," in *ICRA Workshop on Open Source Software*, 2009.

[2] G. Rosu and T. F. Serbanuta, "K overview and simple case study," in *Proceedings of International K Workshop (K'11)*, ser. ENTCS, vol. 304. Elsevier, June 2014, pp. 3–56.

[3] Koord Language Development Team, "A design and analysis framework for distributed cps." [Online]. Available: https://cyphyhouse.github.io/papers/koord.pdf

[4] C. Lattner and V. Adve, "Llvm: A compilation framework for lifelong program analysis & transformation," in *Proceedings of the International Symposium on Code Generation and Optimization: Feedback-directed and Runtime Optimization*, ser. CGO '04. Washington, DC, USA: IEEE Computer Society, 2004, pp. 75–. [Online]. Available: http://dl.acm.org/citation.cfm?id=977395.977673

[5] C. Pinciroli, A. Lee-Brown, and G. Beltrame, "Buzz: An extensible programming language for self-organizing heterogeneous robot swarms," *CoRR*, vol. abs/1507.05946, 2015. [Online]. Available: http://arxiv.org/abs/1507.05946

[6] B. Bohrer, Y. K. Tan, S. Mitsch, M. O. Myreen, and A. Platzer, "Veriphy: Verified controller executables from verified cyber-physical system models," in *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI 2018. New York, NY, USA: ACM, 2018, pp. 617–630. [Online]. Available: http://doi.acm.org/10.1145/3192366.3192406

[7] D. Zufferey, "The REACT language for robotics," 2017.

[8] B. C. Williams, M. D. Ingham, S. H. Chung, and P. H. Elliott, "Model-based programming of intelligent embedded systems and robotic space explorers," *Proceedings of the IEEE*, vol. 91, no. 1, pp. 212–237, 2003.

[9] A. Nordmann, N. Hochgeschwender, and S. Wrede, *A Survey on Domain-Specific Languages in Robotics*. Cham: Springer International Publishing, 2014, pp. 195–206. [Online]. Available: http://dx.doi.org/10.1007/978-3-319-11900-7_17

[10] M. Campusano and J. Fabry, "Live robot programming: The language, its implementation, and robot API independence," *Science of Computer Programming*, vol. 133, pp. 1 – 19, 2017.

[11] S. Gao, S. Kong, and E. M. Clarke, *dReal: An SMT Solver for Nonlinear Theories over the Reals*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 208–214. [Online]. Available: http://dx.doi.org/10.1007/978-3-642-38574-2_14

[12] C. Fan, B. Qi, S. Mitra, and M. Viswanathan, "DryVR: Data-driven verification and compositional reasoning for automotive systems," in *Computer Aided Verification (CAV)*, July 2017.

[13] D. Pickem, P. Glotfelter, L. Wang, M. Mote, A. Ames, E. Feron, and M. Egerstedt, "The robotarium: A remotely accessible swarm robotics research testbed," in *2017 IEEE International Conference on Robotics and Automation (ICRA)*, May 2017, pp. 1699–1706.

[14] B. Bohrer, V. Rahli, I. Vukotic, M. Völp, and A. Platzer, "Formally verified differential dynamic logic," in *Proceedings of the 6th ACM SIGPLAN Conference on Certified Programs and Proofs*, ser. CPP 2017. New York, NY, USA: ACM, 2017, pp. 208–221. [Online]. Available: http://doi.acm.org/10.1145/3018610.3018616

[15] H. Kress-Gazit, G. E. Fainekos, and G. J. Pappas, "Temporal-logic-based reactive mission and motion planning," *IEEE transactions on robotics*, vol. 25, no. 6, pp. 1370–1381, 2009.

[16] M. Kloetzer and C. Belta, "A fully automated framework for control of linear systems from temporal logic specifications," *IEEE Transactions on Automatic Control*, vol. 53, no. 1, pp. 287–297, 2008.

[17] T. Wongpiromsarn, U. Topcu, and R. M. Murray, "Receding horizon control for temporal logic specifications," in *Proceedings of the 13th ACM international conference on Hybrid systems: computation and control*. ACM, 2010, pp. 101–110.

[18] T. Wongpiromsarn, U. Topcu, N. Ozay, H. Xu, and R. M. Murray, "Tulip: a software toolbox for receding horizon temporal logic planning," in *Proceedings of the 14th international conference on Hybrid systems: computation and control*. ACM, 2011, pp. 313–314.

[19] A. Ulusoy, S. L. Smith, X. C. Ding, C. Belta, and D. Rus, "Optimality and robustness in multi-robot path planning with temporal logic constraints," *The International Journal of Robotics Research*, vol. 32, no. 8, pp. 889–911, 2013.

[20] B. Nitzberg and V. Lo, "Distributed shared memory: a survey of issues and algorithms," *Computer*, vol. 24, no. 8, pp. 52 –60, aug. 1991.

[21] S. V. Adve and K. Gharachorloo, "Shared memory consistency models: A tutorial," *IEEE Computer*, vol. 29, pp. 66–76, 1996.

[22] B. Calder, J. Wang, A. Ogus, N. Nilakantan, A. Skjolsvold, S. McKelvie, Y. Xu, S. Srivastav, J. Wu, H. Simitci, J. Haridas, C. Uddaraju, H. Khatri, A. Edwards, V. Bedekar, S. Mainali, R. Abbasi, A. Agarwal, M. F. u. Haq, M. I. u. Haq, D. Bhardwaj, S. Dayanand, A. Adusumilli, M. McNett, S. Sankaran, K. Manivannan, and L. Rigas, "Windows azure storage: A highly available cloud storage service with strong consistency," in *Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles*, ser. SOSP '11. New York, NY, USA: ACM, 2011, pp. 143–157. [Online]. Available: http://doi.acm.org/10.1145/2043556.2043571

[23] "The Apache Cassandra project." http://cassandra.apache.org/, 2017.

[24] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Vosshall, and W. Vogels, "Dynamo: Amazon's highly available key-value store," in *Proceedings of Twenty-first ACM SIGOPS Symposium on Operating Systems Principles*, ser. SOSP '07. New York, NY, USA: ACM, 2007, pp. 205–220. [Online]. Available: http://doi.acm.org/10.1145/1294261.1294281

[25] A. Desai, V. Gupta, E. Jackson, S. Qadeer, S. Rajamani, and D. Zufferey, "P: Safe asynchronous event-driven programming," *SIGPLAN Not.*, vol. 48, no. 6, pp. 321–332, Jun. 2013. [Online]. Available: http://doi.acm.org/10.1145/2499370.2462184

[26] C. Drăgoi, T. A. Henzinger, and D. Zufferey, "Psync: A partially synchronous language for fault-tolerant distributed algorithms," in *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, ser. POPL '16. New York, NY, USA: ACM, 2016, pp. 400–415. [Online]. Available: http://doi.acm.org/10.1145/2837614.2837650

[27] C. Muñoz, V. Carreño, and G. Dowek, *Formal Analysis of the Operational Concept for the Small Aircraft Transportation System*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2006, pp. 306–325.

[28] M. A. Brady and M. O'Kelly, "The official home of f1/10." [Online]. Available: http://f1tenth.org/

[29] B.-S. Chen and R. T. Yeh, "Formal specification and verification of distributed system." MARYLAND UNIV COLLEGE PARK DEPT OF COMPUTER SCIENCE, Tech. Rep., 1983.

[30] G. Rosu and T. F. Serbanuta, "K overview and simple case study," in *Proceedings of International K Workshop (K'11)*, ser. ENTCS, vol. 304. Elsevier, June 2014, pp. 3–56.

[31] Mace, Jonathan and Lee, Jihoon and Toris, Russell and Alexander, Brandon and Bertam, David and Gruhler, Matthias, "Rosbridge suite."

[32] ArduPilot Development Team, "Ardupilot."

[33] Vladimir Ermakov, "mavros."